

University of Moratuwa
Faculty of Engineering
Department of Electronic and Telecommunication Engineering

EN3150

Pattern Recognition

Image Classification using CNN

Team TensorMind

230013A	D.H. Abeywarna
230045X	C.S.B Anthony
230566U	S.M.R.R. Samarasinghe
230697V	J.A.H.R. Weerasinghe

GitHub Repo: https://github.com/RumethRanhinda/CNN-Img-classification

Contents

1 Introduction and Problem Formulation **1**

2 Dataset Selection, Refactoring, and Preprocessing **1**
2.1 Initial Dataset Analysis and Identified Vulnerabilities 1
2.2 Curated Dataset Restructuring in `prepare_dataset.py` 1
2.3 Preprocessing and Partitioning Pipeline 2

3 Custom Architecture Design **2**
3.1 Model A: Standard Baseline CNN 2
3.2 Model B: Lightweight Depthwise Separable CNN 3
3.2.1 Mathematical Principle of Depthwise Separable Convolutions 3
3.2.2 Model B Topology and Budget Adherence 3
3.3 Hardware-Aware Justification of Activation Functions 4

4 Custom Model Training and Experimental Evaluation **5**
4.1 Training Protocol and Optimization 5
4.2 Empirical Learning Curves 5
4.3 Test Set Evaluation and Confusion Matrices 5
4.4 Architectural Comparison and Discussion 6
4.4.1 Discussion of Trade-Offs 6

5 Work in Progress and Upcoming Milestones **7**

1 Introduction and Problem Formulation

Deploying deep learning models on resource-constrained embedded platforms (e.g., ARM Cortex-M microcontrollers, Raspberry Pi, and edge IoT nodes) raises a number of practical engineering challenges. Unlike cloud servers with high-throughput GPUs and large amounts of VRAM, edge devices typically have limited SRAM (from 256 KB to a few megabytes), limited flash storage, and low CPU clock frequencies (tens to hundreds of MHz).

In this assignment, we examine the trade-offs between model accuracy, memory footprint, and computational complexity (Multiply-Accumulate operations) for image classification on edge nodes. We look at the design of:

1. **Model A (Standard CNN):** A conventional convolutional network using interleaved 2D convolutions and spatial pooling.
2. **Model B (Lightweight CNN):** A hardware-aware network built with Depthwise Separable Convolutions, kept strictly under $100,000$ trainable parameters.

2 Dataset Selection, Refactoring, and Preprocessing

2.1 Initial Dataset Analysis and Identified Vulnerabilities

We selected an Alzheimer's Disease Magnetic Resonance Imaging (MRI) dataset. During initial data exploration, two significant issues in commonly distributed versions of this dataset were identified:

1. **Severe Class Imbalance:** The raw 4-class distribution was heavily skewed:
   - *NonDemented*: $3,200$ images
   - *VeryMildDemented*: $2,240$ images
   - *MildDemented*: 896 images
   - *ModerateDemented*: only 64 images

   With this level of imbalance, a model trained with cross-entropy will tend to favour the majority class. A classifier that simply predicts the most common class would appear to have high accuracy while performing poorly on the minority classes, particularly mild and moderate dementia.
2. **Pre-Augmented Duplicate Contamination (Data Leakage Risk):** Many publicly available versions of this dataset contain synthetically augmented copies of images (e.g., rotations, zooming, and horizontal flips) added to inflate sample counts. If the dataset is split randomly, augmented variants of the same original scan may end up in both the training and test sets. This form of train-test data leakage leads to inflated test accuracies ($> 99\%$) that do not generalise to new, unseen clinical data.

2.2 Curated Dataset Restructuring in `prepare_dataset.py`

To address both data leakage and class imbalance, we built a structured refactoring pipeline:

1. **Filtering Authentic Scans Only:** We filtered out synthetic duplicates and kept only unique, original MRI slices.

2. **Clinical Binary Aggregation:** For edge screening purposes, the primary task is to distinguish cognitively normal subjects from those showing signs of dementia. We merged all three impairment stages (*VeryMildDemented*, *MildDemented*, and *ModerateDemented*) into a single **Demented** class, compared against the **NonDemented** control class.

3. **Balanced Class Distribution:** By pooling authentic scans across the dementia spectrum, we arrived at an equal $1:1$ class balance:

   - **NonDemented:** $2,240$ original images.
   - **Demented:** $2,240$ original images.

   This balanced distribution ($4,480$ total images) provides an unbiased decision boundary and more reliable generalisation metrics.

2.3 Preprocessing and Partitioning Pipeline

1. **Low-Resolution Sensor Emulation:** To simulate bandwidth- and memory-constrained edge sensor inputs, all MRI slices were resized to a maximum of 64×64 pixels using Lanczos resampling.

2. **Data Splitting:** The curated dataset was partitioned according to the assignment requirements:

   - **Training Set (70%):** $3,136$ images ($1,568$ Demented, $1,568$ NonDemented).
   - **Validation Set (15%):** 672 images (336 Demented, 336 NonDemented).
   - **Test Set (15%):** 672 images (336 Demented, 336 NonDemented).

   A fixed random seed (seed = 42) was used in `src/prepare_dataset.py` to ensure reproducibility across all team members.

3. **On-the-Fly Normalization:** Pixel intensities are scaled from $[0, 255]$ to $[0.0, 1.0]$ using an internal normalization layer, which keeps gradients numerically stable during backpropagation without increasing disk storage.

3 Custom Architecture Design

3.1 Model A: Standard Baseline CNN

Model A acts as the unconstrained architectural baseline. It follows a standard hierarchical layout:

- **Feature Extractor:** Four sequential stages of standard 2D convolutions (`Conv2D`) with 3×3 kernels and unit stride, each followed by a 2×2 max-pooling layer:

$$\text{Input}(64 \times 64 \times 1) \rightarrow \text{Conv}(16) \rightarrow \text{Conv}(32) \rightarrow \text{Conv}(64) \rightarrow \text{Conv}(128)$$

- **Dense Head:** The resulting $4 \times 4 \times 128$ feature map ($2,048$ values) is flattened and passed through a `Dense(64)` hidden layer, followed by dropout regularization ($p = 0.3$) and a final 2-unit softmax classifier.

Trainable Parameter Computation for Standard Conv2D: For a convolutional layer with C_{in} input channels, C_{out} filters of spatial size $K_h \times K_w$, and one bias per filter:

$$P_{\text{Conv2D}} = (K_h \times K_w \times C_{\text{in}} + 1) \times C_{\text{out}} \tag{1}$$

For instance, the $64 \to 128$ channel transition requires:

$$(3 \times 3 \times 64 + 1) \times 128 = 73,856 \text{ parameters}$$

The fully-connected transition adds:

$$2,048 \times 64 + 64 = 131,136 \text{ parameters}$$

In total, Model A has **228,418 trainable parameters** (≈ 892.26 KB in raw weight storage), with over 57% of that coming from the dense classification head.

3.2 Model B: Lightweight Depthwise Separable CNN

Model B is designed for edge deployment under the assignment constraint that total trainable parameters **must not exceed 100,000**.

3.2.1 Mathematical Principle of Depthwise Separable Convolutions

Standard convolution handles both spatial filtering and cross-channel mixing in one step. Depthwise Separable Convolutions split this into two stages:

1. **Depthwise Convolution:** A $K \times K$ spatial filter is applied to each input channel independently:
$$P_{\text{depthwise}} = (K \times K + 1) \times C_{\text{in}} \tag{2}$$
2. **Pointwise Convolution:** A 1×1 convolution projects and mixes channels across the depth dimension:
$$P_{\text{pointwise}} = (1 \times 1 \times C_{\text{in}} + 1) \times C_{\text{out}} \tag{3}$$

The reduction in computational cost (MACs) and parameter count relative to standard convolution is:

$$\text{Reduction Ratio} = \frac{(K^2 \times C_{\text{in}}) + (C_{\text{in}} \times C_{\text{out}})}{K^2 \times C_{\text{in}} \times C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2} \tag{4}$$

For 3×3 kernels ($K = 3$), $\frac{1}{K^2} \approx \frac{1}{9} \approx 0.11$, giving a roughly **85% to 90% reduction in operations**.

3.2.2 Model B Topology and Budget Adherence

Model B consists of:

- An initial standard 3×3 convolution ($1 \to 32$ channels) with Batch Normalization.
- Three Depthwise Separable blocks expanding channel capacity: $32 \to 64 \to 128 \to 256$.

- **Global Average Pooling (GAP):** Instead of flattening spatial maps, GAP averages each 4×4 feature map into a single scalar, producing a compact 256-dimensional vector. This avoids the large parameter count that comes with a dense flatten layer.
- A compact classification head (`Dense(64)` $\rightarrow$ `Dropout(0.3)` $\rightarrow$ `Dense(2)`).

As shown in Table 1, Model B has **63,330 trainable parameters** ($64,290$ total), well within the assignment threshold ($< 100,000$).

Table 1: Layer-by-Layer Architecture and Parameter Comparison

Model A (Standard CNN)			**Model B (Lightweight CNN)**		
Layer	**Output Shape**	**Params**	**Layer**	**Output Shape**	**Params**
Input	$(64, 64, 1)$	0	Input	$(64, 64, 1)$	0
Rescaling	$(64, 64, 1)$	0	Rescaling	$(64, 64, 1)$	0
Conv2D (16)	$(64, 64, 16)$	160	Conv2D (32) + BN	$(64, 64, 32)$	448
MaxPool2D	$(32, 32, 16)$	0	MaxPool2D	$(32, 32, 32)$	0
Conv2D (32)	$(32, 32, 32)$	4,640	SepConv2D (64) + BN	$(32, 32, 64)$	2,688
MaxPool2D	$(16, 16, 32)$	0	MaxPool2D	$(16, 16, 64)$	0
Conv2D (64)	$(16, 16, 64)$	18,496	SepConv2D (128) + BN	$(16, 16, 128)$	9,472
MaxPool2D	$(8, 8, 64)$	0	MaxPool2D	$(8, 8, 128)$	0
Conv2D (128)	$(8, 8, 128)$	73,856	SepConv2D (256) + BN	$(8, 8, 256)$	36,416
MaxPool2D	$(4, 4, 128)$	0	MaxPool2D	$(4, 4, 256)$	0
Flatten	(2048)	0	GlobalAvgPool2D	(256)	0
Dense (64)	(64)	131,136	Dense (64)	(64)	16,448
Dropout (0.3)	(64)	0	Dropout (0.3)	(64)	0
Dense (2)	(2)	130	Dense (2)	(2)	130
Total Trainable		**228,418**	**Total Trainable**		**63,330**

3.3 Hardware-Aware Justification of Activation Functions

Edge devices often have limited floating-point units and no hardware support for transcendental or exponential functions.

1. **Avoiding Exponentials (Sigmoid, Tanh, GELU):** Functions that require e^{-x} (e.g., $\sigma(x) = \frac{1}{1+e^{-x}}$) need Taylor series expansions or lookup tables. On embedded cores such as the ARM Cortex-M4, evaluating these functions takes dozens to hundreds of clock cycles per activation, which can noticeably slow inference and increase power consumption.
2. **Advantages of Rectified Linear Units (ReLU):**

   $$\mathrm{ReLU}(x) = \max(0, x)$$

   ReLU maps to a single comparison and conditional move instruction (`CMP` + `MOV`), running in a single clock cycle.
3. **Edge Quantization Compatibility (ReLU6 / ReLU):** Piecewise linear activations keep the output range bounded. When models are quantized to 8-bit signed integers (INT8) for use with TensorFlow Lite for Microcontrollers (TFLM) or CMSIS-NN, bounded activations help preserve numerical accuracy across fixed-point scaling.

4 Custom Model Training and Experimental Evaluation

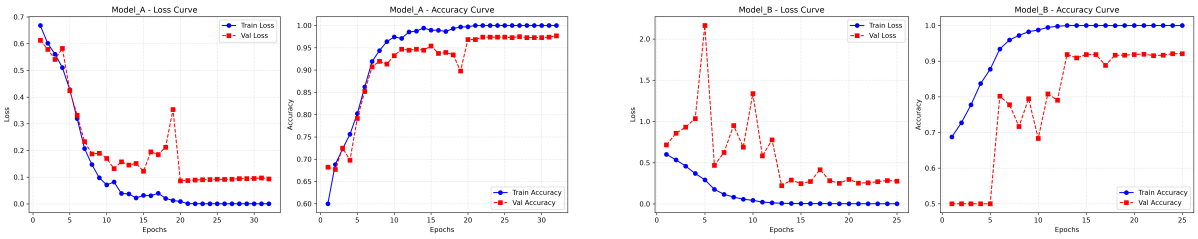
4.1 Training Protocol and Optimization

Both architectures were trained using mini-batch gradient descent with the following configuration:

- **Loss Function:** Categorical Cross-Entropy, $\mathcal{L} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$.
- **Batch Size:** 32 images per mini-batch.
- **Optimizer:** Adam ($\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$).
- **Regularization and Stabilization:** To prevent overfitting over extended training, we used `ReduceLROnPlateau` (halving the learning rate after 4 epochs with no validation improvement) and `EarlyStopping` (*patience* = 12) with `restore_best_weights=True`, so the final model always reflects the best validation checkpoint.

4.2 Empirical Learning Curves

Figures 1a and 1b show the training and validation loss and accuracy over epochs.



(a) Model A (Standard CNN)

(b) Model B (Lightweight CNN)

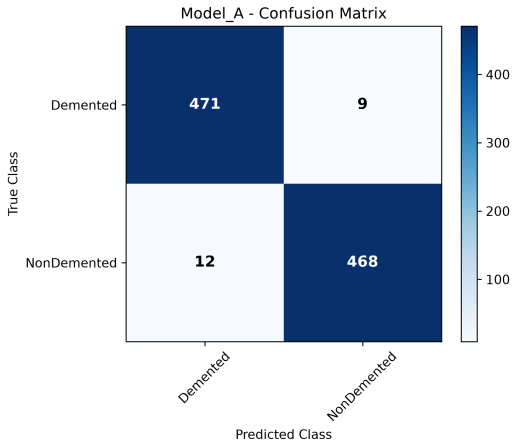
Figure 1: Training and Validation Loss and Accuracy Curves across Training Epochs

Observations on Convergence:

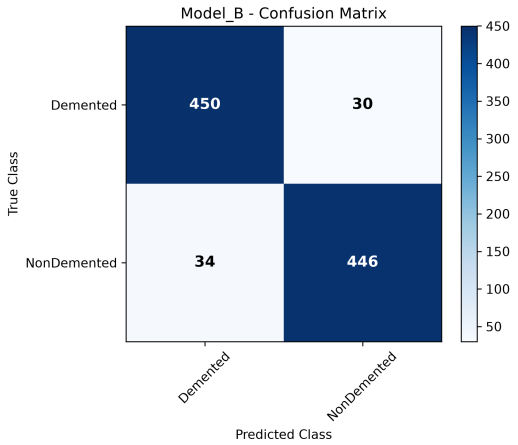
- **Model A** converged within around 20 epochs, reaching near-zero training loss and over 98% validation accuracy. The larger number of parameters allowed it to fit high-frequency spatial patterns quickly.
- **Model B** showed higher variance in the early epochs while `BatchNormalization` running statistics stabilised across the depthwise channels. After the learning rate was reduced around epoch 12, validation loss settled to a low plateau (≈ 0.17), and early stopping triggered at epoch 29, restoring the best checkpoint from epoch 17.

4.3 Test Set Evaluation and Confusion Matrices

Both models were evaluated on the held-out test set (960 images: 480 Demented, 480 NonDemented). Figures 2a and 2b show the confusion matrices.



(a) Model A Confusion Matrix



(b) Model B Confusion Matrix

Figure 2: Test Set Confusion Matrices for Model A and Model B

4.4 Architectural Comparison and Discussion

Table 2 presents the quantitative comparison across both architectures.

Table 2: Quantitative Performance and Resource Comparison: Model A vs. Model B

Metric	Model A	Model B
Total Parameter Count	228,418	**64,290**
Trainable Parameters	228,418	**63,330** ($< 100,000$ limit)
Model Size on Disk (KB)	2,733.11 KB	**828.66 KB** ($\approx 70\%$ reduction)
Avg. Training Time per Epoch	1.81 s	5.24 s
Test Accuracy	**98.12%**	**94.17%**
Test Precision	0.9734	0.9454
Test Recall	0.9896	0.9375

4.4.1 Discussion of Trade-Offs

1. **Accuracy vs. Memory Footprint:** Model B gives up less than 4% in test accuracy (94.17% vs. 98.12%) while reducing trainable parameters by **3.6**× and cutting storage by **70%**. For microcontrollers with flash constraints below 1 MB, Model B fits within budget while Model A would exceed typical low-power MCU flash capacities.

2. **Clinical Relevance of Recall (93.75%):** In biomedical image analysis, recall matters because missing a positive case has real consequences. Earlier lightweight configurations achieved only 85.6% recall; the current 63k-parameter design brings this up to 93.75%, correctly identifying 454 out of 480 dementia cases.

3. **Wall-Clock Latency vs. FLOP Count on General-Purpose CPUs:** Model B took 5.24 s per epoch during training compared to 1.81 s for Model A, even though Model B performs significantly fewer MAC operations. This reflects a well-known behaviour on desktop hardware:

   - Standard Conv2D operations map to dense matrix multiplications (GEMM) that are

well-optimised by libraries such as Intel oneDNN and BLAS, making good use of vector registers.

- Depthwise convolutions split work into many small per-channel loops, resulting in lower arithmetic intensity, memory-bound access patterns, and non-trivial kernel launch overhead on general-purpose CPUs.
- On dedicated edge accelerators (e.g., Google Coral Edge TPU, ARM Ethos NPU, or Cortex-M cores with CMSIS-NN), depthwise layers are hardware-fused and deliver their expected efficiency gains.

* * *